Chapitre 1-
Introduction aux bases de données

—

Master 1
GL et I2A

Contenu

1. Rappels sur les concepts fondamentaux: modèle relationnel, SGBD, langages de requêtes.
2. Types de bases de données: relationnelles, NoSQL, objets, etc.
3. Architecture d'un SGBD: client-serveur, architectures distribuées.
4. Concepts de transaction, concurrence, et récupération de données.

Section 1-

Rappels sur les concepts fondamentaux: modèle relationnel, SGBD, langages de requêtes.

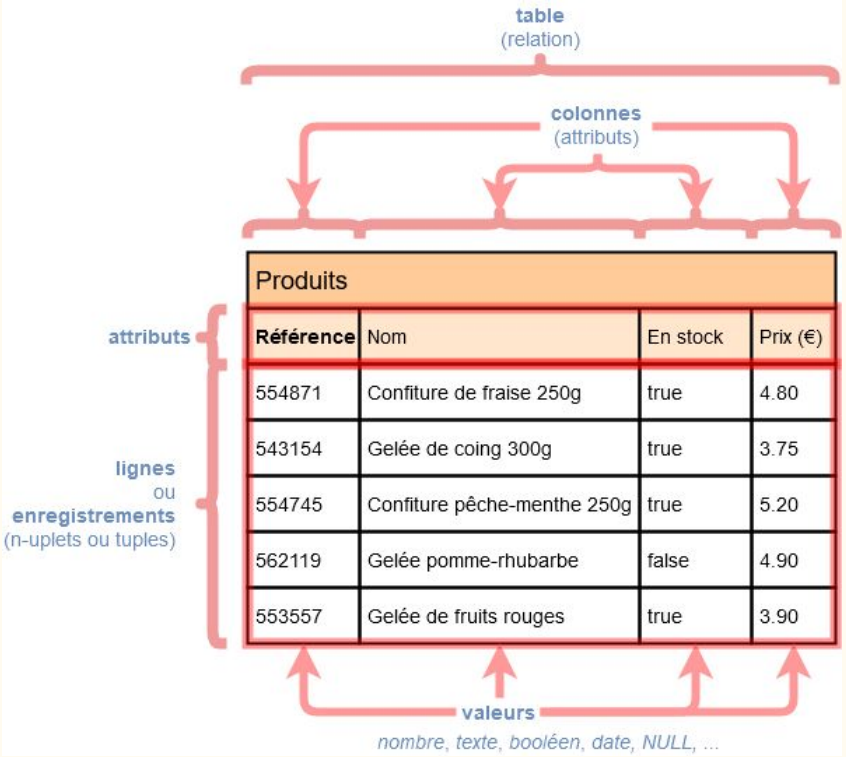
Plan du cours

1. Le modèle relationnel
2. Systèmes de Gestion de Base de Données (SGBD)
3. Langages de requêtes

Le Modèle Relationnel

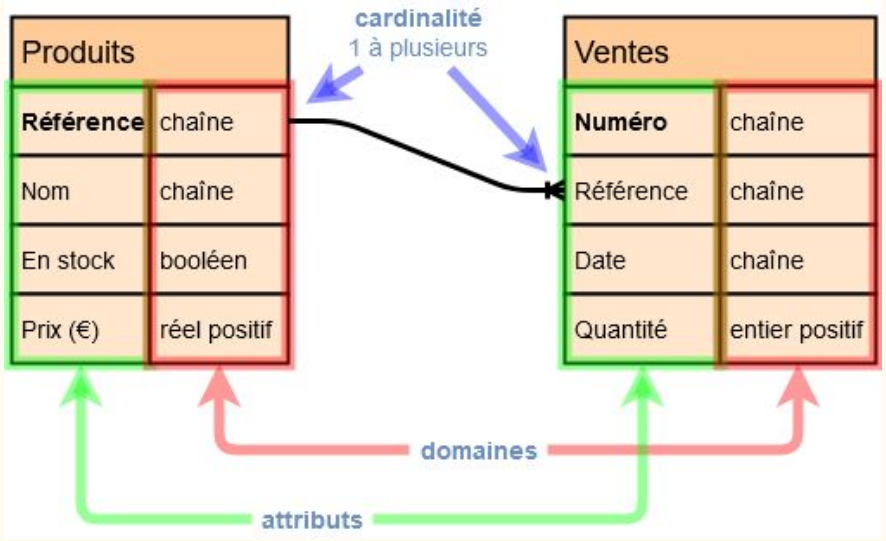
- Défini par Edgar F. Codd en 1970
- Données représentées sous forme de tables (relations)
- Propriétés :
  - Atomicité des valeurs
  - Unicité des lignes
  - Ordre des lignes et colonnes non significatif

Exemple de table



Produits

Référence	Nom	En stock	Prix (€)
554871	Confiture de fraise 250g	true	4.80
543154	Gelée de coing 300g	true	3.75
554745	Confiture pêche-menthe 250g	true	5.20
562119	Gelée pomme-rhubarbe	false	4.90
553557	Gelée de fruits rouges	true	3.90



Système de Gestion de Base de Données (SGBD)

- Logiciel permettant de stocker, organiser et manipuler des données
- Rôles principaux :
  - Définir les données
  - Manipuler les données
  - Sécurité et contrôle d'accès
  - Intégrité et cohérence
- Types :
  - Relationnels (MySQL, PostgreSQL...)
  - Orientés objets
  - NoSQL (MongoDB, Cassandra...)

Langages de requêtes

- **LDD (DDL) :**
  - CREATE,
  - ALTER,
  - DROP

Exemple : CREATE TABLE Etudiants(...)

- **LMD (DML) :**
  - INSERT,
  - UPDATE,
  - DELETE,
  - SELECT

Exemple : SELECT Nom, Prenom FROM Etudiants WHERE Filiere='Informatique'

- **LCD (DCL) :**
  - GRANT,
  - REVOKE

Cycle d'exécution d'une requête SQL

Section 2-

Types de bases de données: relationnelles, NoSQL, objets, etc.

Bases de données relationnelles (RDB)

- Principe :
  - données organisées en tables (relations).
- Caractéristiques :
  - Tables avec lignes (tuples) et colonnes (attributs).
  - Utilisation du langage SQL.
  - Intégrité référentielle (clés primaires et étrangères).
- Avantages :
  - Structure rigoureuse.
  - Transactions fiables (ACID).
- Exemples :
  - MySQL, PostgreSQL, Oracle, SQL Server.

Bases de données orientées objet (OODB)

- Principe :
  - intégration des concepts de la programmation orientée objet dans la BD.
- Caractéristiques :
  - Données = objets (avec attributs + méthodes).
  - Supporte l'héritage, le polymorphisme.
  - Stockage direct des objets complexes (images, documents, multimédia).
- Avantages :
  - Cohérence entre modèle objet et stockage.
  - Adaptées aux applications scientifiques, multimédia, CAO/DAO.
- Exemples :
  - ObjectDB, Versant ODB, db4o.

Exemple de OODB

```
class Utilisateur {
    String nom;
    String email;
    Liste<Media> medias;   // Association : un utilisateur possède plusieurs médias

    void ajouterMedia(Media m);
}
```

```
class Media {
    String titre;
    Date dateAjout;
}
```

```
class Image extends Media {
    int largeur;
    int hauteur;
    String format;
}
```

```
class Video extends Media {
    int duree;
    String resolution;
}
```

Exemple d’objets stockés

Utilisateur 1 :

- nom = "Alice",
- email = "alice@mail.com”
- medias = [Image("vacances.jpg"), Video("concert.mp4")]

Utilisateur 2 :

- nom = "Karim",
- email = "karim@mail.com"
- medias = [Image("selfie.png")]

Avantages

- Pas de transformation Objet-Relationnel (ORM inutile).
- Héritage, encapsulation et polymorphisme gérés directement.
- Idéal pour applications multimédia, CAO/DAO, simulations.

Exemples de SGBD orientés objet

- db4o,
- ObjectDB,
- Versant,
- GemStone.
- Certains SGBD hybrides relationnel-objet (PostgreSQL, Oracle avec objets).

Bases de données NoSQL

- Principe :
  - “Not Only SQL”, conçues pour gérer de très grandes quantités de données non structurées ou semi-structurées.
- Catégories principales :
  - Clé-Valeur (Redis, DynamoDB) → rapidité d’accès.
  - Documentaires (MongoDB, CouchDB) → stockage JSON-like.
  - Colonnes larges (Cassandra, HBase) → données massives distribuées.
  - Graphes (Neo4j, OrientDB) → relations complexes entre données.
- Avantages :
  - Flexibilité : pas besoin de schéma rigide (contrairement au relationnel).
  - Scalabilité horizontale : peut gérer des millions de documents répartis sur plusieurs serveurs.
- Inconvénients :
  - Moins strictes que les BD relationnelles sur la cohérence.

Exemples

Collection: Produits

```
{
  "_id": "p123",
  "nom": "Ordinateur Portable",
  "marque": "Dell",
  "prix": 1200,
  "caractéristiques": {
    "cpu": "Intel i7",
    "ram": "16 Go",
    "stockage": "512 Go SSD"
  },
  "categories": ["Informatique", "PC"],
  "stock": 15
}
```

```
{
  "_id": "p124",
  "nom": "Smartphone",
  "marque": "Samsung",
  "prix": 900,
  "caractéristiques": {
    "ram": "8 Go",
    "stockage": "256 Go"
  },
  "categories": ["Informatique", "Téléphonie"],
  "stock": 30
}
```

Exemples

Collection: Clients

```
{
  "_id": "c001",
  "nom": "Alice",
  "email": "alice@mail.com",
  "commandes": [
    { "produit_id": "p123", "quantité": 1, "date": "2025-09-28" },
    { "produit_id": "p124", "quantité": 2, "date": "2025-09-29" }
  ]
}
```

Exemple

MongoDB stocke un étudiant ainsi :

```json
{
  "Matricule": 101,
  "Nom": "Benali",
  "Prenom": "Samir",
  "Filiere": "Informatique"
}
```

Autres types de bases de données

- Bases hiérarchiques : données organisées en arborescences (ex : IMS d'IBM).
- Bases en réseau : relations complexes via pointeurs.
- Bases orientées colonnes : optimisées pour l'analytique (ex : BigQuery, Vertica).
- Bases multimédias : conçues pour gérer des images, vidéos, sons.
- Bases temps réel / temps série : adaptées aux capteurs, IoT (ex : InfluxDB, TimescaleDB).

Base de données hiérarchiques

Le modèle hiérarchique est un des premiers modèles de bases de données (utilisé par IMS de IBM dans les années 1960). Les données sont organisées sous forme d'arbre :

- Chaque enregistrement est appelé nœud.
- Chaque nœud a un seul parent (sauf la racine).
- La relation est de type 1 → N (un parent peut avoir plusieurs enfants).

Exemple: Gestion d'une bibliothèque

- Bibliothèque est la racine.
- Chaque Livre peut avoir plusieurs Exemplaires et être lié à un ou plusieurs Auteurs.
- Chaque Abonné peut avoir plusieurs Emprunts.

```
Bibliothèque
 ├── Livre
 |    ├── Auteur
 |    └── Exemplaire
 |
 └── Abonné
      └── Emprunt
```

- Avantage : très rapide pour naviguer de la racine vers les enfants.
- Inconvénient : rigidité (les relations N–N sont difficiles à gérer).

Base de données réseau

- Développé dans les années 1960 (notamment avec CODASYL DBTG).
- Les données sont organisées sous forme de graphes (nœuds et liens), avec des enregistrements (records) reliés par des ensembles (sets).
- Modèle très efficace pour gérer des relations complexes (plusieurs-à-plusieurs) avec des accès rapides,
- Mais il est moins flexible et plus difficile à manipuler que le modèle relationnel.

Exemple: Gestion d'une bibliothèque

- **Entités principales :**
  - Livre (ISBN, titre, auteur)
  - Abonné (id, nom, adresse)
  - Emprunt (date_emprunt, date_retour)

- **Organisation en réseau :**
  - Un Livre peut être lié à plusieurs Emprunts.
  - Un Abonné peut être lié à plusieurs Emprunts.
  - Les relations sont donc représentées par des ensembles reliant les enregistrements.

Exemple: Gestion d'une bibliothèque

- **Représentation en graphe:**

```
[Abonné]—(fait)—>[Emprunt]<—(concerne)—[Livre]
```

- Chaque ensemble relie un propriétaire (owner) et un membre (member).
  - Ensemble "fait" : Abonné (owner) → Emprunt (member)
  - Ensemble "concerne" : Livre (owner) → Emprunt (member)

- **Exemple d'enregistrements:**
  - Livre : (ISBN=123, Titre="SGBD", Auteur="Date")
  - Abonné : (id=1, Nom="Sara")
  - Emprunt : (date_emprunt=01/10/2025, date_retour=15/10/2025)
- **Relations en réseau :**
  - Sara fait un Emprunt
  - Cet Emprunt concerne le Livre "SGBD"

Bases de données orientées Colonnes

- Systèmes de gestion conçus pour stocker les données par colonne plutôt que par ligne, contrairement aux SGBDR classiques.
- Principe :
  - Dans une base classique, chaque enregistrement (ligne) est stocké ensemble.
  - Dans une base orientée colonne, toutes les valeurs d'une colonne sont stockées côte à côte.

Bases de données orientées Colonnes

- Optimisées spécialement pour l'analyse de données.
- Avantages:
  - **Lecture rapide :** les requêtes analytiques accèdent souvent à quelques colonnes sur des millions de lignes → lecture plus efficace.
  - **Meilleure compression :** les valeurs d'une même colonne sont souvent similaires → compression plus efficace.
  - **Moins d'I/O :** seules les colonnes nécessaires sont lues en mémoire, ce qui réduit fortement le volume de données à traiter.
  - Excellente scalabilité pour des requêtes massives et complexes.

Bases de données orientées Colonnes

Limite principale :

- Moins performant pour les opérations transactionnelles (OLTP) avec de nombreuses insertions et mises à jour.
- Adapté sur tout pour les requêtes analytiques complexes (OLAP).

Cas d'usage typiques :

- Data warehouse/data lakehouse
- Tableaux de bord BI
- Analyses statistiques ou temps réel sur gros volumes de données.

Google Big Query

amazon REDSHIFT

ClickHouse

opentext™ Analytics Database

Bases de données multimédias

- Conçues pour ***stocker***, ***gérer*** et ***interroger*** des données riches comme les images, vidéos, sons, ou encore des documents complexes (ex. 3D, animations).
- Types de données gérées :
  - 🖼️ Images (photos, illustrations...)
  - 🎥 Vidéos (films, clips, séquences)
  - 📢 Sons (audio, musique, enregistrements)
  - 📄 Documents composites (texte + média)

Bases de données multimédias

- Caractéristiques principales :
  - **Stockage de gros objets (BLOB) :** les médias sont souvent volumineux → stockage optimisé pour de gros fichiers.
  - **Métadonnées :** chaque média est accompagné d'informations descriptives (titre, date, auteur, résolution...) pour faciliter la recherche.
  - **Indexation avancée :** permet des recherches par contenu (ex. couleur dominante d'une image, durée d'une vidéo).
  - **Support de formats variés :** JPEG, MP4, MP3, WAV, PNG, etc.
  - Intégration avec des systèmes de streaming pour la diffusion en continu.

Bases de données multimédias

- Fonctionnalités typiques :
  - Requêtes par métadonnées (par exemple “toutes les vidéos de 2023”).
  - Requêtes basées sur le contenu (par exemple “images similaires à cette photo”).
  - Gestion de droits et contrôle d’accès aux médias.

- Exemples / technologies :
  - Oracle Multimedia (ancien module d’Oracle pour images, sons et vidéos)
  - PostgreSQL avec types BLOB et extensions multimédia
  - Elasticsearch ou solutions NoSQL pour indexation et recherche de médias
  - Systèmes spécialisés de gestion d’actifs numériques (DAM).

Bases de données Temps réel/Temps série

- Conçues pour ***stocker*** et ***analyser*** des données horodatées, c'est-à-dire associées à un instant précis (ex. mesures de capteurs, données IoT, logs, monitoring système...).
- Principe : Chaque enregistrement contient généralement :
  - une valeur (température, tension, etc.),
  - un horodatage précis,
  - et parfois des étiquettes (tags) pour décrire la source (capteur, lieu, etc.).

Bases de données Temps réel/Temps série

- Caractéristiques principales :
  - Insertion rapide de grandes quantités de données en continu (haute fréquence).
  - Stockage optimisé pour des séries temporelles compactes et structurées.
  - Requêtes efficaces sur des intervalles de temps (par jour, heure, minute...).
  - Agrégation et downsampling intégrés (moyenne, max, min, somme...).
  - Gestion automatique de la rétention (suppression des données anciennes selon une politique définie).

Bases de données Temps réel/Temps série

- Cas d'usage typiques :
  - Internet des objets (IoT) : données de capteurs en continu.
  - Supervision industrielle.
  - Monitoring système et performance applicative.
  - Finance (cours boursiers en temps réel).
  - Réseaux électriques, météo, smart cities.
- Exemples:

influxdb

Timescale

Prometheus

Tableau comparatif

Critère	Bases relationnelles	Bases orientées colonnes	Bases multimédias	Bases temps réel / séries temporelles
Modèle de données	Lignes et tables structurées	Colonnes regroupées pour lecture rapide	Objets multimédias + métadonnées	Séries temporelles (valeur + horodatage)
Type d'usage principal	OLTP (transactions)	OLAP (analytique, reporting)	Gestion de médias (images, sons, vidéos)	IoT, monitoring, logs
Performance	Excellente en écriture/lecture transactionnelle	Excellente en lecture analytique	Dépend du format et de l'indexation	Excellente en ingestion temps réel
Indexation / requêtes	SQL complet, jointures	SQL analytique, agrégations rapides	Indexation par contenu ou métadonnées	Requêtes par intervalles de temps
Stockage	Par lignes	Par colonnes	Fichiers volumineux (BLOB) + index	Données horodatées compressées

Tableau comparatif (Suite)

Critère	Bases relationnelles	Bases orientées colonnes	Bases multimédias	Bases temps réel / séries temporelles
Compression	Moyenne	Très bonne	Variable selon format média	Très bonne
Complexité de requêtes	Très riche (SQL standard)	Très riche (analytique)	Spécialisée (contenu/métadonnées)	Axée sur le temps
Gestion du temps réel	Limitée	Limitée	Faible	Excellente
Exemples	MySQL, PostgreSQL, Oracle Database	Google BigQuery, Vertica, Amazon Redshift	Oracle Multimedia, PostgreSQL (BLOB), Elasticsearch	InfluxDB, TimescaleDB, Prometheus
Cas d'usage typiques	Applications métiers, facturation, ERP	Tableaux de bord BI, data warehouse	Plateformes multimédias, archivage, recherche d'images/sons	IoT, supervision, séries temporelles

Tableau comparatif (Suite)

Critère	Bases relationnelles	Bases orientées colonnes	Bases multimédias	Bases temps réel / séries temporelles
Avantages	- Modèle éprouvé et standardisé - Requêtes complexes - Forte cohérence des données	- Très performant en analytique - Compression efficace - Scalabilité horizontale	- Adapté aux fichiers lourds - Recherche par contenu - Bon support des métadonnées	- Ingestion très rapide - Optimisé pour séries temporelles - Agrégations efficaces
Inconvénients	- Moins performant pour gros volumes analytiques - Moins flexible pour données non structurées	- Moins adapté aux transactions - Plus complexe à administrer	- Peu adapté aux transactions - Requêtes limitées hors contenu multimédia	- Requêtes limitées en complexité - Moins adapté aux données relationnelles classiques

Section 3:

Architecture d'un SGBD: client-serveur, architectures distribuées.

Introduction

- Un SGBD n'est pas seulement un ensemble de tables, mais aussi une architecture logicielle qui permet aux utilisateurs et applications d'accéder aux données.
- Deux grandes catégories :
  - Architecture client-serveur.
  - Architectures distribuées.

Architecture Client-Serveur

- **Principe :** séparation entre :
  - Client → interface utilisateur / application.
  - Serveur → gestion et stockage des données (moteur du SGBD).

- **Fonctionnement :**
  - Le client envoie des requêtes SQL au serveur.
  - Le serveur traite la requête et retourne les résultats.

Architecture Client-Serveur

- **Avantages :**
  - Centralisation des données.
  - Sécurité et contrôle d'accès.
  - Maintenance facilitée.

- **Exemples :**
  - MySQL Server avec clients phpMyAdmin, Java, Python.
  - PostgreSQL avec PgAdmin.

Architecture simplifiée d'un SGBD

Architectures distribuées

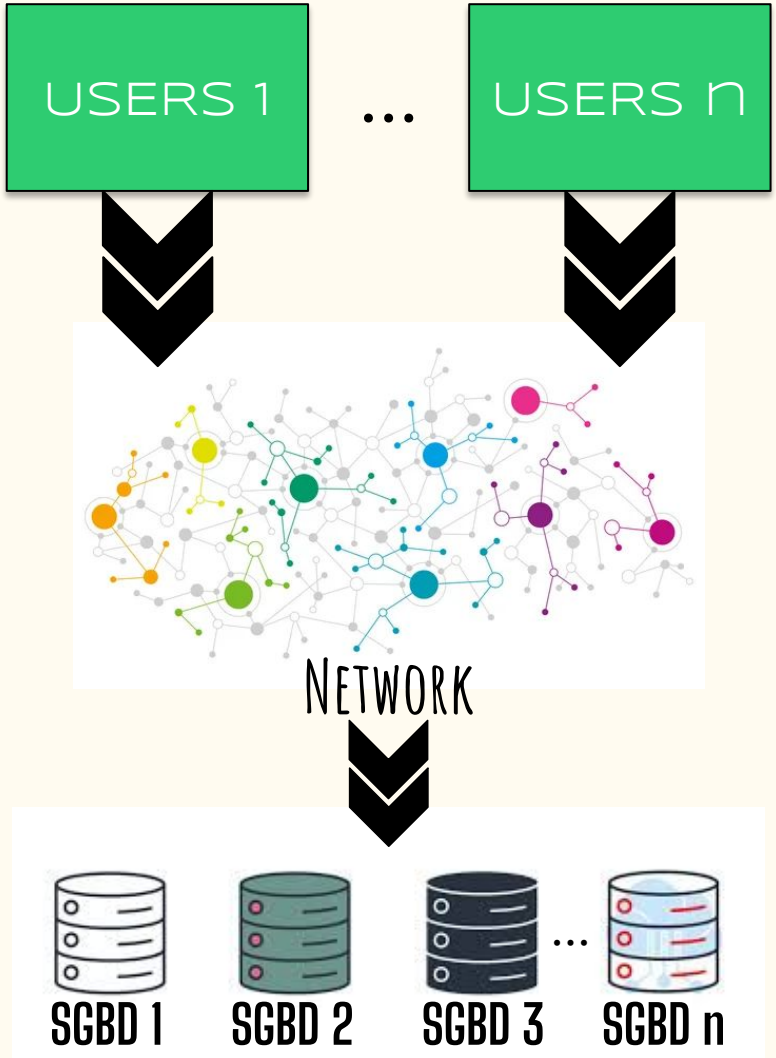
- **Principe :** les données et/ou le traitement sont répartis sur plusieurs machines.

- **Motivations :**
  - Répartition de charge.
  - Résilience aux pannes.
  - Proximité avec l'utilisateur.

- **Types d'architectures distribuées :**
  - Bases réparties → chaque site contient une partie des données.
  - Bases répliquées → copies des données sur plusieurs sites (synchronisation nécessaire).
  - Hybrides → combinaison des deux.

Architectures distribuées

- **Avantages :**
  - Haute disponibilité.
  - Scalabilité horizontale.
- **Inconvénients :**
  - Complexité de gestion (cohérence, synchronisation).



Etude comparative

Critère	Client-Serveur	Distribuée
Centralisation	Oui	Non (plusieurs sites)
Performance	Dépend du serveur central	Scalabilité horizontale
Résilience	Point de panne unique	Tolérance aux pannes
Gestion	Simple	Complexe (réplication, synchro)

Section 4:

Concepts de transaction, concurrence, et récupération de données.

Transaction

Une transaction est une suite d'opérations sur la base de données qui doit être exécutée de façon atomique (tout ou rien).

Exemple: Transfert d'argent (compte A → compte B)

1. Débiter compte A.
2. Créditer compte B.

Ces deux requêtes doivent être exécutées ensemble : si une étape échoue, retour à l'état initial.

```
UPDATE compte SET solde = solde - 200 WHERE id = 1;
UPDATE compte SET solde = solde + 200 WHERE id = 2;
```

Propriétés ACID

Une transaction doit respecter les 4 propriétés ACID :

Propriété	Signification	Description
A – Atomicité	Tout ou rien	Si une étape échoue, toute la transaction est annulée
C – Cohérence	Intégrité maintenue	La transaction laisse la base dans un état cohérent
I – Isolation	Indépendance	Les transactions concurrentes n'interfèrent pas
D – Durabilité	Résultat permanent	Les changements validés sont enregistrés même en cas de panne

Cycle de vie d'une transaction

1. BEGIN / START TRANSACTION → démarre la transaction
2. Exécution des requêtes SQL (INSERT, UPDATE, DELETE...)
3. COMMIT → valider les changements
4. ROLLBACK → annuler les changements (en cas d'erreur)

Cycle de vie d'une transaction

Exemple:

```sql
START TRANSACTION;

UPDATE compte SET solde = solde - 200 WHERE id = 1;
UPDATE compte SET solde = solde + 200 WHERE id = 2;

COMMIT; -- ou ROLLBACK si problème
```

Concurrence

Signifie que plusieurs transactions ou processus s'exécutent en même temps et peuvent accéder aux mêmes données.

Exemple : deux utilisateurs essaient de modifier le même enregistrement en même temps.

Problèmes possibles

- Lecture sale (dirty read)
- Lecture non reproductible
- Phantom read (données fantômes)
- Blocages et interblocages (deadlocks)

Solutions typiques

- Verrouillage (pessimiste)
- Contrôle par version ou horodatage (optimiste)
- Niveaux d'isolation de transaction

Lecture sale (Dirty Read)

- Une lecture sale se produit lorsqu'une transaction lit des données modifiées par une autre transaction non encore validée (non commit).
- Si l'autre transaction est annulée (rollback), la première aura lu des données invalides.

Étapes	Transaction T1	Transaction T2
1	–	Met solde = 1200 pour le compte 1 (pas encore commit)
2	Lit solde = 1200 (dirty read)	–
3	–	Annule la modification (rollback)

Résultat : T1 a utilisé une valeur qui n'existe plus (valeur incohérente).

Solution : utiliser un niveau d'isolation plus élevé (par exemple Read Committed) pour éviter de lire des données non validées.

Lecture non reproductible (Non Repeatable Read)

- Elle se produit lorsqu'une transaction lit une donnée, puis la relit plus tard et obtient une valeur différente car une autre transaction l'a modifiée entre-temps.

Étapes	Transaction T1	Transaction T2
1	Lit solde = 1000	–
2	–	Met solde = 1200 et commit
3	Relit solde → maintenant 1200	–

Résultat : T1 obtient deux valeurs différentes pour la même donnée au cours de sa transaction.

Solution : utiliser le niveau d'isolation Repeatable Read pour verrouiller les lignes lues.

Lecture fantôme (Phantom Read)

- Elle se produit lorsqu'une transaction relit une requête avec une condition (ex. WHERE) et obtient plus ou moins de lignes qu'avant, car une autre transaction a inséré ou supprimé des données correspondantes

Étapes	Transaction T1	Transaction T2
1	Execute SELECT * FROM comptes WHERE solde > 1000 → retourne 3 lignes	–
2	–	Insère un nouveau compte avec solde = 1500 et commit
3	Relit la même requête → retourne maintenant 4 lignes	–

Résultat : des "lignes fantômes" apparaissent dans le résultat de T1.

Solution : utiliser le niveau d'isolation Serializable, qui empêche l'insertion/suppression concurrente sur la plage de données concernée.

Blocage et interblocage (Deadlock)

Blocage simple :

- Quand une transaction T1 verrouille une ressource (par exemple une ligne) et une autre transaction T2 doit attendre que T1 libère cette ressource.

Étapes	Transaction T1	Transaction T2
1	Met à jour le compte 1 ⟶ verrou	–
2	–	T2 veut aussi mettre à jour le compte 1 ⟶ T2 attend T1.

Blocage et interblocage (Deadlock)

Interblocage (Deadlock) :

- Quand deux transactions se bloquent mutuellement, chacune attendant une ressource verrouillée par l'autre.

Étapes	Transaction T1	Transaction T2
1	Verrouille compte 1	–
2	–	Verrouille compte 2
3	Demande compte 2 (Attente → bloqué par T2)	Demande compte 1 (Attente → bloqué par T1)

Résultat : T1 et T2 sont bloquées indéfiniment

Solution :

- Le SGBD détecte l'interblocage et annule une transaction.
- Bonne conception des transactions (ordre des accès constant, durées courtes).
- Verrouillage raisonné.

Pour résumer

Problème	Cause principale	Conséquence	Solution typique
Dirty Read	Lecture de données non validées	Lecture de valeurs incohérentes	Read Committed
Non Repeatable Read	Modification de données entre deux lectures	Incohérence de valeur	Repeatable Read
Phantom Read	Insertion/Suppression entre deux lectures	Nombre de lignes change	Serializable
Deadlock	Verrou mutuel circulaire	Blocage total	Détection / rollback / bon ordre d'accès

Gestion de la concurrence

Il existe trois grandes approches classiques pour gérer la concurrence dans les systèmes de bases de données.

- Verrouillage (pessimiste)
- Contrôle par version ou horodatage (optimiste)
- Niveaux d'isolation de transaction

Elles servent à éviter les problèmes comme la lecture sale, la lecture non reproductible, les lectures fantômes ou les interblocages.

Verrouillage (Pessimiste)

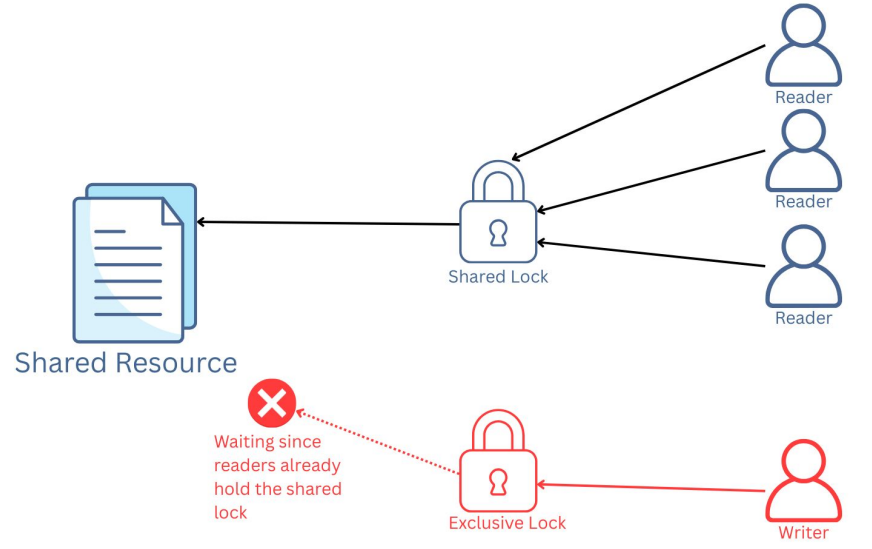
- **Idée générale:**

  Part du principe que des conflits vont arriver, donc elle verrouille les données dès qu'une transaction les utilise pour empêcher les autres d'y accéder de manière incompatible.

- **Comment ça marche:**
  - Quand une transaction lit ou modifie une donnée, le SGBD met un verrou sur cette donnée.
  - Les autres transactions doivent attendre que le verrou soit libéré (après COMMIT ou ROLLBACK).
  - Cela évite les conflits, mais peut ralentir le système si plusieurs transactions attendent.

Verrouillage (Pessimiste)

- **Types:**
  - Verrou partagé (***Shared Lock***) → pour lecture (plusieurs peuvent lire en même temps).
  - Verrou exclusif (***Exclusive Lock***) → pour écriture (une seule transaction à la fois).



Verrouillage (Pessimiste)

- **Avantages :**
  - Très sûr (évite la plupart des anomalies).
  - Cohérence forte des données.
- **Inconvénients :**
  - Moins de parallélisme.
  - Risque de blocages et interblocages.

Exemple :

  - T1 veut modifier un solde → verrou exclusif.
  - T2 veut lire ou modifier ce solde → doit attendre T1.

Contrôle par version ou horodatage (Optimiste)

- **Idée générale:**
  - L'approche optimiste part du principe que les conflits sont rares.
  - Elle n'utilise pas de verrou immédiat, mais vérifie à la fin de la transaction s'il y a eu un conflit.
- **Comment ça marche:**
  - Chaque donnée a un horodatage ou une version.
  - Une transaction lit/modifie localement les données.
  - Lors du COMMIT, le SGBD vérifie si quelqu'un d'autre a modifié les mêmes données entre-temps.
    - Si non ✅ → commit accepté.
    - Si oui ⚠️ → la transaction échoue ou est relancée.

Verrouillage (Pessimiste)

- **Avantages :**
  - Très bon parallélisme (pas de verrou pendant l'exécution).
  - Performant quand peu de conflits.
- **Inconvénients :**
  - En cas de conflit, la transaction doit être annulée et recommencée.
  - Moins efficace si beaucoup de conflits.

- **Exemple :**
  - T1 et T2 lisent la même donnée (version 1).
  - T1 modifie et commit → version 2.
  - T2 essaie de commit → détecte que la version a changé → rollback.

Niveaux d'isolation des transactions

- C'est un mécanisme essentiel pour gérer la concurrence entre transactions dans un SGBD.
- Déterminent dans quelle mesure les transactions sont protégées des effets des autres transactions en cours.
- Normalisés par la norme SQL ANSI/ISO,
- Implémentés par la plupart des SGBD (ex. PostgreSQL, MySQL, Oracle Database...).
- Se sont au nombre de 04:
  - ***READ UNCOMMITTED***
  - ***READ COMMITTED***
  - ***REPEATABLE READ***
  - ***SERIALIZABLE***

READ UNCOMMITTED

Principe :

- Une transaction peut lire les modifications non encore validées par d'autres transactions.
- Il n’y a presque aucune protection contre les conflits.

Problèmes possibles :

- Lecture sale (dirty read)
- Lecture non reproductible
- Lecture fantôme

Exemple :

- T1 modifie le solde d’un compte mais ne commit pas.
- T2 lit déjà cette valeur → si T1 fait rollback, T2 a lu une donnée invalide.

Utilisation :

- Très rare en pratique (dangereux pour l’intégrité des données).

READ COMMITTED

Principe :

- Une transaction ne peut lire que les données validées (commitées) par d'autres transactions.
- Cela évite les lectures sales, mais les données peuvent encore changer entre deux lectures.

Problèmes possibles :

- Lecture non reproductible
- Lecture fantôme

Exemple :

- T1 lit le prix d'un produit.
- T2 modifie le prix et commit.
- T1 relit le prix → il a changé.

Utilisation :

- Niveau par défaut dans Oracle Database et Microsoft SQL Server. Bon compromis entre sécurité et performance.

REPEATABLE READ

Principe :

- Une transaction voit les mêmes valeurs lorsqu'elle relit une donnée, même si d'autres transactions l'ont modifiée entre-temps.
- Le SGBD verrouille les lignes lues ou utilise des versions stables.

Problèmes possibles :

- Lecture fantôme

Exemple :

- T1 lit la liste des employés avec salaire > 2500.
- T2 ajoute un employé avec salaire 3000 et commit.
- T1 relit la requête → voit plus de lignes.

Utilisation :

- Niveau par défaut dans MySQL (InnoDB).

SERIALISABLE

Principe :

- Isolation maximale : le SGBD simule une exécution séquentielle des transactions.
- Aucun phénomène (dirty, non repeatable ou phantom read) n'est possible.
- Le système peut utiliser des verrous étendus ou le contrôle par version pour assurer cette isolation.

Inconvénients:

- Moins de parallélisme
- Risque plus élevé de blocage ou de rollback

Exemple :

- T1 lit les employés > 2500.
- T2 ne peut pas insérer un nouvel employé dans cette plage tant que T1 n'a pas terminé.

Utilisation :

- Recommandée pour les systèmes critiques (banques, comptabilité...).

Commandes SQL pratiques

- Changer le niveau d'isolation dans une session

```sql
-- Exemple pour PostgreSQL ou MySQL
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
BEGIN;
-- instructions SQL ici
COMMIT;
```

- Voir le niveau courant

```sql
SHOW TRANSACTION ISOLATION LEVEL;
```

Pour résumer

Niveau	Dirty Read	Non Repeatable Read	Phantom Read	Sécurité	Concurrence
READ UNCOMMITTED	✅	✅	✅	Faible	Très forte
READ COMMITTED	❌	✅	✅	Moyenne	Bonne
REPEATABLE READ	❌	❌	✅	Haute	Moyenne
SERIALIZABLE	❌	❌	❌	Très haute	Faible

- Plus le niveau est bas → plus la base est rapide mais risquée.
- Plus le niveau est élevé → plus la base est cohérente mais plus lente.
- Choisir le bon niveau dépend du besoin métier.

Récupération de données

C'est un mécanisme de reprise après sinistre (disaster recovery) et de restauration de la base pour revenir à un état cohérent après une défaillance.

Objectif:

- Ne perdre aucune donnée validée (committed),
- Revenir à un état cohérent de la base,
- Permettre aux transactions interrompues d'être annulées ou rejouées correctement.

Types de pannes possibles

Type de panne	Exemple	Impact
Panne de transaction	Bug dans le programme ou annulation utilisateur	Une seule transaction affectée
Panne système	Crash du SGBD ou du serveur	Transactions en cours perdues
Panne disque / matériel	Disque dur défaillant	Données perdues
Catastrophe (incendie, inondation…)	Site entier hors service	Perte majeure, reprise sur site de secours

Mécanismes utilisés pour la récupération

1- Journalisation (Log des transactions): Le SGBD conserve dans un journal toutes les opérations importantes (écriture, mise à jour, suppression) avant qu'elles soient réellement appliquées à la base.

- Write-Ahead Logging (WAL) : le journal est écrit avant les données → permet de rejouer ou d'annuler les transactions.
- Si panne → le système lit le journal :
  - UNDO pour annuler les transactions incomplètes,
  - REDO pour rejouer celles validées mais non écrites sur disque.

Exemple :

Une transaction T1 débite un compte de 100 DA → si panne avant écriture physique, le journal permet de rejouer ou d'annuler proprement l'opération.

Mécanismes utilisés pour la récupération

2- Points de contrôle (Checkpoints): Le SGBD crée périodiquement des points stables de la base :

- À chaque checkpoint, l'état actuel de la base est enregistré,
- En cas de panne, on ne relit pas tout le journal depuis le début, mais à partir du dernier checkpoint.

Cela accélère énormément la récupération.

Mécanismes utilisés pour la récupération

3- Copies de sauvegarde (Backups): Les sauvegardes sont essentielles :

- Sauvegarde complète : copie intégrale de la base à une date donnée,
- Sauvegarde incrémentale : seules les modifications depuis la dernière sauvegarde.

En cas de panne matérielle ou perte de données, on restaure la sauvegarde la plus récente puis on rejoue le journal pour revenir à l'état exact avant la panne.

Mécanismes utilisés pour la récupération

4- Redondance / réplication (optionnel mais fréquent):
Pour renforcer la sécurité:

- Réplication des données sur un autre serveur ou centre,
- Basculement automatique (failover) si le serveur principal tombe.

Exemple : PostgreSQL propose la réplication en streaming.

Étapes typiques de la récupération après panne

1. Détection de la panne
2. Redémarrage du SGBD en mode recovery
3. Lecture du journal (depuis le dernier checkpoint)
4. UNDO des transactions incomplètes
5. REDO des transactions validées
6. Retour à l'état normal

Bonnes pratiques pour minimiser les pertes

- Faire des sauvegardes régulières (quotidiennes ou automatiques)
- Activer la journalisation des transactions
- Utiliser les checkpoints fréquents
- Tester les procédures de restauration régulièrement
- Avoir un plan de reprise en cas de sinistre majeur

Pour résumer

- La récupération de données dans un SGBD en cas de panne repose principalement sur le ***journal des transactions***, ***les points de contrôle*** et ***les sauvegardes***.
- Le but est de revenir à un ***état cohérent*** sans ***perte de transactions*** validées.

Fin du chapitre 1